AOE2 CIV EXPLORER

Code documentation

What each part of the code does, and why

22 October 2026 · 19 chapters · 258 functions

This document explains the code of AoE2 Civ Explorer. Each file is a chapter. In the script chapters, each function defined at the top level of the script is a subchapter, saying what it does and why in at most 240 characters. Helpers defined inside a function are part of that function's text.

“Exported” marks a function other scripts use; “internal” marks one only its own script uses. Rules and requirements are those of RULES.md and REQUIREMENTS.md. The two JSON chapters at the end are kept short.

Contents

1	index.html	10 parts
2	style.css	12 parts
3	script.js	1 part
4	js/datahandler.js	40 functions
5	js/model.js	23 functions
6	js/stringbuilder.js	11 functions
7	js/nodetree.js	24 functions
8	js/optionshandler.js	15 functions, 1 object
9	js/pictures.js	5 functions
10	js/report.js	3 functions
11	js/tests/run.js	7 functions
12	js/tests/helpers.js	57 functions
13	js/tests/static.js	17 functions
14	js/tests/behaviour.js	18 functions
15	js/tests/interface.js	11 functions

16	js/tests/pictures.js	13 functions
17	js/tests/audit.js	14 functions
18	data.json	
19	locales/de.json (example)	

1 index.html

The page's structure, and all of its HTML (Rule 2). It holds no text of its own: the scripts fill in every word in the chosen language.

1.1 Head

Sets the language, character set, viewport, light and dark colour schemes, title and description, and loads style.css and script.js as a module. The tab icon starts as the empty data:, so no picture loads before the scripts run (Req. 33).

1.2 Header

Holds the title twice: the full sentence of Req. 2 for Simple UI, and “Civ Explorer” for the pictures (Req. 0). Then the cogwheel Options button (Req. 6), a text character with a hidden label. Hidden until the page starts.

1.3 Messages

A noscript line for browsers without JavaScript, and a load error shown if data.json cannot be loaded, so a visitor is never left with a blank page.

1.4 Tree pane

The left pane, in the order of Req. 3: heading, Clear, Expand all and Collapse all, the Find field with its label, clearing cross and live match count, the must have and must lack legend, and the empty tree.

1.5 Civilisations pane

The right pane (Req. 4): heading, live count, the Filters list (Req. 22), the civilisation list and the hint shown when it is empty. It scrolls on its own, so the page never does (Rule 12).

1.6 Footer

Names and links the data source and its date (Req. 5), then Microsoft's notice, in English, with a link to its rules. The code writes the sentence around the links.

1.7 Options window

A dialog (Req. 7, 8, 32): title, the language label, button and list, Simple UI, Include Unique and Universal with its description, Include Regionals, Civ view, Tooltips, and Close.

1.8 Report window

A dialog for the reports of the tests and the updater: title, live status, result list, download links and Close. Always in English, so it is marked lang="en".

1.9 Templates

Markup for tree rows, Filters buttons, civilisations and their icons, the tree's icons, language options, report lines and downloads. The code copies these instead of writing HTML (Rule 2).

1.10 Tooltip

The note that follows the mouse with the pictures on, over the tree, a civilisation or Include. Hidden otherwise.

2 style.css

How the page looks, and all of its CSS (Rule 2): colours, sizes and layouts for big and small screens, in light and dark mode, with the pictures on or off.

2.1 Colours

Named colours for light mode, redefined for dark mode: text, lines, marks, hover, Find's yellow, the green and red of the marks, and the dark box behind the white dash. The tests measure their contrast.

2.2 Base

Box sizing, a full-height page that never scrolls (Rule 12), the system font with 1.5 line spacing (WCAG 1.4.8), a 3 px focus outline (WCAG 2.4.13), and a class that hides text from sight but not from screen readers.

2.3 Pictures

Every rule for the pictures starts with .pictures, so Simple UI turns them all off (Rules 9, 22): parchment, cursors, frames, buttons, checkbox sheets, icons and tiles, and the short title, Find first and no legend (Req. 0).

2.4 Fonts

Georgia, Times New Roman Bold and Trajan Pro Bold from assets/fonts/, under their own names, trying the installed copy first. Only rules for the pictures use them, so Simple UI never downloads a font.

2.5 Header and Options

Lays out the title and cogwheel, and the Options window as a grid of fixed height, so opening the language list never resizes it (Rule 18). The current language is marked with a bar and bold.

2.6 Panes

Two panes side by side from 720 px wide; below that they share a phone's height, and on short screens everything scrolls as one, as at 400% zoom (WCAG 1.4.10). Each pane's head stays while its list scrolls.

2.7 Controls

Buttons and the Find field are at least 44 by 44 px (WCAG 2.5.5), with visible borders and a hover colour. Find's message keeps its column, so the box never changes size while typing (Rule 18).

2.8 Tree

Indented rows with dotted guide lines, 44 px rows, turning arrows, pressed rows, bold top-level and marked names, and found rows in yellow with the light colours (Req. 20), or amber with the pictures. Counts never wrap.

2.9 Check boxes

With Simple UI, marks drawn with borders (Req. 14, 23, 26): a green tick, a red cross, and a white dash on a dark box, in system colours when forced. With the pictures, the sheet's own rows, and a black dash.

2.10 Filters and list

Filters buttons as rounded chips, and the civilisation list in columns that never split a name, or as tiles with the pictures.

2.11 Dialogs

Size, padding, border and backdrop of the Options and report windows; with the pictures, the Options window's frame and parchment, measured to fit around the same window.

2.12 Test helper

The wider text spacing the tests apply (WCAG 1.4.12). Used only while testing.

3 script.js

Where the page starts, in the main folder. It defines no functions; its top-level code runs once.

3.1 Start-up

Sets Simple UI from the saved choice first, so pictures load at once or never. Loads data.json and the visitor's language (all of them for ?test or ?update), then starts the page; ?test loads the tests, ?update the updater.

4 js/datahandler.js

Everything about data: loading, the chosen language and its text, sets of civilisations, saved choices, and the updater.

4.1 `attempt(action)` EXPORTED

Runs a function and returns null instead of an error. Used where the browser may refuse, such as storage in a private window, so a refusal never breaks the page.

4.2 `recall(key)` EXPORTED

Reads a saved choice from the browser's storage, or null. Remembers the visitor's language and options between visits.

4.3 `remember(key, value)` EXPORTED

Saves a choice in the browser's storage, or removes it when the value is null. Failures are ignored, as the page works without storage.

4.4 `load(url)` EXPORTED

Fetches a file as text and returns a promise. It uses XMLHttpRequest because, unlike fetch, it also reads files when the page is opened from disk, as the tests in CI do.

4.5 `fill(template, values)` EXPORTED

Replaces {name} placeholders in a text with values, leaving unknown ones alone. All text lives in the data with placeholders, so the code never builds sentences itself.

4.6 `treeKeys(nodes, key)` EXPORTED

Lists every item (or group) name in a tree, top to bottom. Used to check the tree, to find groups that need text, and by the updater to know which items to look up.

4.7 `useLanguage(code)` EXPORTED

Switches the active language: its text, language tag, sorting, plural rules and number format. Everything that shows, sorts or counts text reads these, so one call changes them all.

4.8 `nameOf(item)` EXPORTED

Returns a tree item's or group's name in the current language, or its key if missing. Used wherever an item is shown or searched.

4.9 `civName(civ)` EXPORTED

Returns a civilisation's name in the current language, or its key if missing. Keeps the civilisation list translated.

4.10 `iconFile(picture)` EXPORTED

Returns the path of a tree item's icon from its kind and picture number, such as assets/images/icons/nodes/unit/8.png. Used to show icons, and in the updater's report.

4.11 `plural(key, n)` EXPORTED

Picks the right form of a counted text, such as “1 civilisation” or “5 civilisations”, by the language's plural rules, and fills in the formatted number.

4.12 `plain(text)` EXPORTED

Lower-cases a text and strips accents, spaces and punctuation, so Find matches names however they are typed, in any language.

4.13 `localeFile(code)` EXPORTED

Returns the path of a language's file, such as locales/de.json. Keeps the pattern in one place for loading and for the updater's downloads.

4.14 `unpack(file)` EXPORTED

Turns data.json as stored into the shape used in memory, where overrides and text are kept per language, so other languages can be added beside English.

4.15 `addLanguage(set, code, file)` EXPORTED

Adds one language file's overrides and text to the data in memory. Used when a language loads, and when a test reads back the files the updater would write.

4.16 `useData(raw)` EXPORTED

Reads data.json and gives each civilisation one bit, so a set of civilisations is a single number that & and | combine quickly.

4.17 `loadLanguage(code)` EXPORTED

Loads a language's file unless it is loaded already. A visitor downloads only the language shown, not all 17.

4.18 `loadLanguages()` EXPORTED

Loads every language's file. The tests and the updater need them all.

4.19 `pack(set)` EXPORTED

Splits the data in memory back into files: data.json with English, and one file per other language. The updater writes these, and a test checks that nothing is lost.

4.20 `preferredLanguage()` EXPORTED

Chooses the language to start in: the one in the address (?lng=), else the visitor's saved choice, else English (Req. 10). Shared links and return visits keep their language; first visits are English.

4.21 `bit(civ)` INTERNAL

Returns a civilisation's bit, or stops with an error for an unknown name, so a typo in data.json cannot silently drop a civilisation.

4.22 `size(set)` EXPORTED

Counts the civilisations in a set by counting its bits. Behind every number on the page.

4.23 `setOf(rule)` EXPORTED

Turns an availability rule from data.json (“all”, a has list or a lacks list) into a set. Rules stay short in the file while the code works with sets.

4.24 `civsOf(set)` EXPORTED

Turns a set back into a list of civilisations, in data.json order. Used to list the civilisations that fit.

4.25 `isScalar(value)` INTERNAL

Tells whether a value is text, a number, true or false, or null. Helps toJson() decide what fits on one line.

4.26 `isFlatList(value)` INTERNAL

Tells whether a value is a list of scalars only, which toJson() writes on one line.

4.27 `flat(value)` INTERNAL

Writes a value as one line of JSON, with a space after each comma and colon. The building block of toJson().

4.28 `toJson(value, indent)` EXPORTED

Writes JSON the way data.json is laid out: short objects and lists on one line, the rest indented. Keeps the updater's files readable and their changes easy to review.

4.29 `isLower(char)` INTERNAL

Tells whether a character is a lower-case letter. joinLines() uses it to tell a hyphenated word from a real hyphen.

4.30 `capitalise(text)` INTERNAL

Upper-cases the first letter of a text, as some names in the game start in lower case.

4.31 `joinLines(text, settings)` EXPORTED

Joins a game text broken with
 into one line, by the language's rules for spaces and hyphens, and tidies spaces. The game breaks names to fit its buttons.

4.32 `localName(strings, node, settings, offset)` INTERNAL

Finds an item's name in a language's game texts. If that name looks abbreviated, it takes the bold name from the item's help text instead, which is written out in full.

4.33 `availabilityOf(owners, civs)` EXPORTED

Writes who has an item in its shortest form: “all”, a has list or a lacks list. Keeps data.json small; a test checks that every rule is stored this way.

4.34 `pictureOf(node, counts)` EXPORTED

Picks an item's icon for data.json from aoe2techtree.net's data: its kind, the picture most civilisations show, and whether it is common, regional or unique, which Include Regionals uses.

4.35 `missingText(set)` EXPORTED

Lists, per language, the page texts and group names that are missing. A test and the updater's report use it, so no language shows a blank.

4.36 `gather(civs, trees)` INTERNAL

Reads aoe2techtree.net's tree of every civilisation into three tables: each item's node, the civilisations that have it, and how often each of its pictures is used. The updater's first step.

4.37 `translate(current, fresh, parts)` INTERNAL

Writes every civilisation's and item's name in every language from the game's texts, unless an override says otherwise, writing out abbreviated names from the help text.

4.38 `compare(current, fresh, parts)` INTERNAL

Lists every change for the updater's report: civilisations added or gone, items whose owners or icon changed, renamed items, items not yet in the tree, and missing texts.

4.39 `refresh(current, techtree, trees, strings)` INTERNAL

The updater's core: works out the civilisations, who has what, the icons and every name from aoe2techtree.net's data, and the report. Stops if the tree names an unknown item, or one twice.

4.40 `fetchUpdate()` EXPORTED

Downloads aoe2techtree.net's data, trees and texts, runs refresh(), and returns the report and the files that changed. So a game update needs only a browser.

5 js/model.js

The tree and its rules: which items are shown, which are marked, and how a mark passes down a line. It never touches the page.

5.1 `setIncluded(on)` EXPORTED

Sets whether unique and universal items are included. A function, because other scripts cannot change an imported variable themselves.

5.2 `setRegionals(on)` EXPORTED

Sets whether regional items are included, for the same reason.

5.3 `collect(nodes, parent)` INTERNAL

Turns the tree from data.json into item objects with their set of civilisations, path, parent and children, in tree order. The page and the tests work with these.

5.4 `whole(item)` EXPORTED

Returns the civilisations that have an item and everything under it, remembering the result. It is the second number on a closed item.

5.5 `prune(nodes, every, regionals)` INTERNAL

Returns the tree to show: without unique groups unless every is set, without regional items unless regionals is, and without items every civilisation or only one has, unless something under them stays.

5.6 `rebuild()` EXPORTED

Builds the items again from the tree prune() gives, keeping each item's mark by its path. Runs at start and when an Include box changes.

5.7 `setMark(item, mark)` EXPORTED

Sets a mark on an item and on everything under it. Used by Clear, by marking a building or group, and to clear an item with all below it.

5.8 `state(item)` EXPORTED

Returns the state of an item with everything under it: “has”, “lacks”, none, or “mixed” when they differ. Decides the dash in a box, and what a click on a building does.

5.9 `isUnitOrTech(item)` EXPORTED

Tells whether an item sits under another item, which makes it a unit or technology rather than a building or group. Marking and naming work differently for the two (Req. 24 to 31).

5.10 `isTop(item)` INTERNAL

Tells whether an item is the top of its line: a unit or technology straight under a building or group, such as Militia or Knight. Its mark passes to the whole line (Req. 27).

5.11 `descendants(item)` INTERNAL

Lists every item under an item. Used to pass marks down, and to look at the marks below.

5.12 `passedTo(item)` INTERNAL

Returns an item and the only children below it that its mark passes to: the whole line from the top of a line (Req. 27), else only while they have as many civilisations (Req. 29, 30).

5.13 `mark(item, kind)` EXPORTED

Toggles a mark. Buildings and groups pass it to all in them, units and technologies down their line as passedTo() says, and never up. Returns the items to show (Req. 28, 31).

5.14 `boxState(item)` EXPORTED

Returns what an item's box shows. A marked unit or technology shows its own mark, and a dash only if items below carry another mark (Req. 26). Any other item shows state().

5.15 `fit()` EXPORTED

Works out which civilisations fit every mark: having each item marked must have and lacking each marked must lack. The heart of the civilisation list and of every number. Returns the marked items.

5.16 `markOf(item)` EXPORTED

Returns the mark an item stands for in Filters: the state of everything under it if that is uniform, or else its own mark.

5.17 `lineEnd(item)` EXPORTED

Returns the last item of a line of only children, or nothing if the item branches. Used for marking lines and naming their filters.

5.18 `lineOf(item)` EXPORTED

Returns the whole line an item belongs to, from its top to its last item, or just the item. Filters shows one button per line.

5.19 `chainOf(item)` EXPORTED

Returns an item and each only child below it, down to the first fork or the end. Opening or marking an item opens this chain (Req. 28).

5.20 `filtersOf()` EXPORTED

Works out the Filters buttons: one per marked item whose parent does not already say the same, and one per line and kind of mark. Keeps Filters short.

5.21 `share()` EXPORTED

Finds names that more than one item in the tree uses, so chipName() can add the parent's name and keep Filters buttons clear.

5.22 `chipName(item, kind)` EXPORTED

Names a Filters button: “Full” and the name for a fully marked root, building or group with something under it (Req. 24); the deepest must-have item of a unit's or technology's line (Req. 25).

5.23 `unmarkLine(item, kind)` EXPORTED

Clears the marks behind a Filters button: those of its kind in its line, or the item and all under it. A must lack mark in the line survives clearing must have.

6 js/stringbuilder.js

The page's text: the fixed texts, the texts of the chosen language, the tree's names, the list of civilisations and Filters, and the two small helpers every script that touches the page uses.

6.1 `$(id)` EXPORTED

Short for document.getElementById, used wherever the page is touched.

6.2 `clone(id)` EXPORTED

Copies the element inside a template in index.html. Repeated markup lives in index.html (Rule 2); the code only fills it in.

6.3 `simple()` EXPORTED

Tells whether Simple UI is on, from the pictures class on the page, which is what shows the pictures. One source of truth, readable from every script.

6.4 `compose(node, template, values)` EXPORTED

Writes a text with placeholders into an element, putting elements such as the source link where their placeholder is. Lets a translated sentence contain a link.

6.5 `writeFixed()` EXPORTED

Writes the texts that never change: the report window's English words, the source's and Microsoft's links, and Microsoft's notice around its link. Runs once at start.

6.6 `writeTexts()` EXPORTED

Writes every text of the chosen language: the page's lang, each element marked data-text, and the footer's source and date (Req. 5, 11).

6.7 `label()` EXPORTED

Writes every tree item's name in the current language, and finds names used more than once.

6.8 `civIcon(civ)` INTERNAL

Makes a civilisation's icon from its template, with its file from icons in data.json. Only used with the pictures on.

6.9 `listCivs(marked)` EXPORTED

Lists the civilisations that fit the marks, translated and sorted (Req. 4), with icons and links to aoe2techtree.net when the pictures are on; writes the count, and the hint when the list is empty.

6.10 `listFilters()` EXPORTED

Makes one Filters button per filter (Req. 22, 23), with its mark and name, and a label telling screen readers what a click clears. Hides Filters when nothing is marked.

6.11 `showLoadError(shown)` EXPORTED

Shows, or hides, the message that the data or a language could not be loaded.

7 js/nodetree.js

The tree on the left: drawing its rows, opening and closing them, their counts, marking with clicks, touch and keys, Find, and the buttons above the tree.

7.1 `isOpen(item)` EXPORTED

Tells whether an item's branch is open, read from its arrow's aria-expanded, which is the one place this is kept.

7.2 `shown(item)` EXPORTED

Tells whether an item's row can be seen, that is, whether every branch above it is open.

7.3 `itemAt(target)` EXPORTED

Returns the tree item an element belongs to, so one listener on the tree serves every row, and the tooltip knows when it is over the tree.

7.4 `dress(item)` EXPORTED

Gives a tree row its icon between its box and name, from data.json, or takes it away with Simple UI on (Rules 9, 22). The tree is drawn, and Simple UI switched, through it.

7.5 `build(list, ul)` INTERNAL

Makes a row for each item from the template, with its arrow, and ids that tie the arrow and the list to the name for screen readers. Calls itself for the items below.

7.6 `endPress()` INTERNAL

Cancels a long press in progress and its highlight, when the finger moves or lifts, or the tree is drawn again.

7.7 `plant()` INTERNAL

Draws the tree from the model again, keeping open branches open, and dresses each row. Runs at start and through replant().

7.8 `count(item)` INTERNAL

Writes an item's count of civilisations that fit the filters: its own number, or “own–all” when closed with items under it, and a spoken version for screen readers.

7.9 `setOpen(item, open)` EXPORTED

Opens or closes an item's branch, and rewrites its count, since a closed item shows two numbers.

7.10 `expand(item)` INTERNAL

Opens an item and each only child below it, so opening Militia shows its whole line in one step. Used by the arrows and the Right key.

7.11 `paint()` INTERNAL

Updates every box: its state for the eye, aria-checked for screen readers, and the hidden “must lack” words.

7.12 `update()` EXPORTED

Redraws all that depends on the marks: boxes, the civilisation list and its count, the hint, every item's count and Filters. Runs after every change.

7.13 `toggle(item, kind)` INTERNAL

Handles a mark from a click, a key, a right-click or a long press: calls mark(), opens the way to every row it reached (Req. 28, 31), and redraws.

7.14 `reveal(node)` INTERNAL

Scrolls the tree's own list just enough to show a row, and nothing around it, so the Find box being typed in never moves (Rule 18).

7.15 `find()` EXPORTED

Runs Find: highlights the matching items in yellow, announces how many there are, opens the branches down to them and shows the first (Req. 20, 21).

7.16 `replant()` EXPORTED

Builds the tree again and redraws it, its names and its counts, and runs Find again if it holds text. Used when an Include box changes.

7.17 `onKey(e)` INTERNAL

Handles keys in the tree: Space marks must have and X must lack (Req. 3); Up, Down, Home and End move; Right and Left open, enter, leave and close branches (WCAG 2.1.1).

7.18 `onPress(e)` INTERNAL

Starts a right-click, or a touch that becomes a long press after half a second, which marks must lack (Req. 3). The row shows as pressed when the long press is ready.

7.19 `onRelease(e)` INTERNAL

Ends a press: marks the item must lack if the press was ready and ends on the same item, and swallows the click that follows.

7.20 `onMenu(e)` INTERNAL

Stops the browser's own menu over the tree, and marks must lack on the menu key. Right-clicks and long presses are left to onPress() and onRelease().

7.21 `onClick(e)` INTERNAL

Opens or closes a branch from its arrow, or marks an item must have (Req. 3), unless the click ends a long press.

7.22 `onChip(e)` INTERNAL

Clears the marks behind a Filters button, and moves focus to the next button, or to the heading after the last one (WCAG 2.4.3).

7.23 `wireTools()` INTERNAL

Connects Expand all, Collapse all and Clear (Req. 17 to 19), Find with its clearing cross and Enter, and the Filters buttons. Runs once at start.

7.24 `startTree()` EXPORTED

Draws the tree and connects it: keys, presses, moves that cancel a long press, the menu and clicks, then the tools around it. Runs once at start.

8 js/optionshandler.js

The Options window and the choices in it: the language menu, the two Include boxes, Simple UI and the tab icon. It also starts the page with the visitor's saved choices.

8.1 `options()` EXPORTED

Lists the language menu's buttons. Used by the menu's keys and by the tests.

8.2 `setMenu(open, focus)` EXPORTED

Opens or closes the language menu, hiding the settings while it is open so the window keeps its size (Rule 18), and moves focus if asked.

8.3 `buildLanguages()` INTERNAL

Makes one button per language, named and tagged in its own language, so screen readers say each name right (WCAG 3.1.2). Runs once at start.

8.4 `setLanguage(code)` EXPORTED

Shows the page in a language: its texts, the menu's name and marked choice, the tree's names, then redraws, and runs Find again if it holds text. Runs at start and when a language is chosen.

8.5 `tick(id, on)` EXPORTED

Shows an Options box as ticked or not, for the eye and for screen readers.

8.6 `setInclude(on)` EXPORTED

Switches Include Unique and Universal (Req. 13 to 15): saves the choice, ticks the box and builds the tree again, keeping marks.

8.7 `setRegional(on)` EXPORTED

Switches Include Regionals: saves the choice, ticks the box and builds the tree again, keeping marks.

8.8 `showIcon()` INTERNAL

Gives the tab its icon from data.json with the pictures on, and the empty icon data:, with Simple UI, so Simple UI uses no image at all (Req. 33).

8.9 `setSimple(on)` EXPORTED

Switches Simple UI (Req. 32, 33): saves the choice, ticks the box, adds or removes the pictures class, sets the tab icon, and dresses the tree's rows and the list again.

8.10 `wireOptions()` INTERNAL

Connects the cogwheel button, the Options window and its boxes, by click and by Space. Runs once at start.

8.11 `choose(code)` INTERNAL

Chooses a language: loads its file if needed, shows it, saves the choice, and puts it in the address (?lng=), so a shared link keeps it.

8.12 `onMenuKey(e)` INTERNAL

Handles the language menu's keys: Down and Up open it at the current language and move through it, Home and End jump, and Escape closes it and returns focus (WCAG 2.1.1).

8.13 `onScrollbar(e)` INTERNAL

Tells whether a click landed on the language list's own scroll bar, which Chrome reports as a click, so that scrolling the list does not close it.

8.14 `wireLanguages()` INTERNAL

Connects the language menu: its button, a click on a language or anywhere else in the list (Req. 11, 12), its keys, and closing when focus or a click goes elsewhere. Runs once at start.

8.15 `start()` EXPORTED

Starts the page: restores the Include choices, draws the tree, builds the language menu, writes the fixed texts and the tab icon, connects every control, shows the chosen language, and reveals the page.

8.16 `flip` EXPORTED OBJECT

An object holding each Options box's action by its id, so one click and one key listener serve every box. js/pictures.js adds Tooltips to it.

9 js/pictures.js

What only the pictures need: Civ view, list or tiles, and the tooltip.

9.1 `setView(choice)` EXPORTED

Shows the civilisations as a list or as tiles, remembers the choice and marks it among the radio buttons. Tiles are pictures, so CSS shows them only with the pictures on.

9.2 `setTips(on)` EXPORTED

Switches the tooltips on or off, remembers the choice, and ticks the Tooltips box.

9.3 `tipFor(target)` INTERNAL

Returns the tooltip's text for what the mouse is over: the marking hint over the tree, Include's description over Include, or a civilisation's address.

9.4 `wireTip()` INTERNAL

Moves the tooltip with the mouse, into the Options window while it is open, at the cursor's corner and inside the window. It shows only for a mouse, with the pictures and Tooltips on.

9.5 `startPictures()` EXPORTED

Restores Civ view and Tooltips from the saved choices, connects the radio buttons and the Tooltips box, and starts the tooltip. Runs once, after start().

10 js/report.js

The report window of the updater and the tests.

10.1 `openReport(title, status)` EXPORTED

Opens the report window, used by the tests and the updater, with a title and a status line, and empties it.

10.2 `report(line)` EXPORTED

Adds a line to the report window.

10.3 `runUpdate()` EXPORTED

Runs the updater: shows its progress, lists what changed, and offers each changed file for download. Opened with ?update; it changes nothing by itself.

11 js/tests/run.js

Runs the tests and shows the report. Loaded only when the address ends in ?test.

11.1 `merge(where)` INTERNAL

Combines the results into one line per test, failed if any of its checks failed, with the first details and the window's size. A test may check in several states and languages.

11.2 `reportLine(result, tools)` INTERNAL

Writes a test's report line, such as “Test 15 failed: …”, from its number, name and details.

11.3 `loadFailed()` EXPORTED

Marks the tests as failed when the page could not load, so CI never waits for a result that cannot come.

11.4 `prepare()` INTERNAL

Notes the visitor's language, options and saved choices, then sets the tests' starting point: Include unticked, Include Regionals ticked, Simple UI and Tooltips on.

11.5 `restore(before)` INTERNAL

Puts back the visitor's language, options, address and saved choices after the tests.

11.6 `rounds()` INTERNAL

Runs every test in three rounds, functional, accessibility and maintainability, in the order TESTS.md lists them, picking sample items afresh where the tree was rebuilt.

11.7 `runTests()` EXPORTED

Runs the tests, numbers them in the order they first ran, checks the 100-test limit (Rule 17), marks the page passed or failed for CI, and shows the report.

12 js/tests/helpers.js

Small tools the test files share: recording results, finding elements, acting like a visitor, and picking sample items.

12.1 `check(name, ok, detail)` EXPORTED

Records one test result. Every test reports this way, and each name is one line in the report.

12.2 `expect(name, offenders)` EXPORTED

Records a test that passes when its list of offenders is empty, naming the first few, so a failure points at the exact items.

12.3 `wait(ms)` EXPORTED

Waits a number of milliseconds. Used to hold a long press.

12.4 `settled(img)` INTERNAL

Waits until an image has loaded or failed.

12.5 `loaded(images)` EXPORTED

Waits until every image has loaded or failed, or ten seconds have passed, so the tests check icons only once they are there.

12.6 `phone()` EXPORTED

Tells whether the window uses the phone layout: narrower than 720 px and at least 640 px tall.

12.7 `width(selector)` INTERNAL

Measures the width of the first element a selector finds.

12.8 `spans()` EXPORTED

Tells whether the footer is as wide as the panes, or hidden in the phone layout.

12.9 `need(ok, problem)` EXPORTED

Returns a problem in a list if a condition fails, and an empty list if it holds, so a test can list exactly what went wrong.

12.10 `same(a, b)` EXPORTED

Tells whether two lists are equal, item by item. Used to compare lists of civilisations.

12.11 `codes()` EXPORTED

Lists the language codes from data.json.

12.12 `ui()` EXPORTED

Returns the page's texts in the current language.

12.13 `box(el)` EXPORTED

Short for an element's getBoundingClientRect(): its place and size on the screen.

12.14 `keys()` EXPORTED

Lists every item in the full tree.

12.15 `describe(el)` EXPORTED

Names an element briefly, such as button#clear or a tree row's text, for failure messages.

12.16 `rendered(el)` EXPORTED

Tells whether an element can be seen: laid out, not hidden, and not text for screen readers only. The accessibility checks look at what is rendered.

12.17 `everything()` EXPORTED

Lists every element that can be seen on the page.

12.18 `styles(el)` EXPORTED

Gets an element's styles and those of its ::before, ::after and ::marker parts, so decorations are checked too.

12.19 `pictured(el)` EXPORTED

Tells whether an element shows a picture through CSS: a background, list marker, border image, content or cursor, in itself or its parts.

12.20 `tabIcons()` EXPORTED

Lists the tab icons the page asks for, leaving out the empty icon data:, which is no picture.

12.21 `pictures()` EXPORTED

Lists every picture on the page: images, tab icons and everything CSS draws with a picture, the page's own background included. Simple UI must leave it empty (Req. 33).

12.22 `picturesOn()` EXPORTED

Tells whether the main pictures show: the page's background, the Options button's picture and the tab icon.

12.23 `rgb(colour)` INTERNAL

Reads the numbers of a CSS colour.

12.24 `yellow(colour)` EXPORTED

Tells whether a colour is yellow: red and green high and close, blue clearly lower (Req. 20).

12.25 `tone(colour)` INTERNAL

Names a colour as white, green, red or other, loosely enough for light and dark colours.

12.26 `drawn(box)` EXPORTED

Names the mark drawn on a checkbox, such as “green tick”, “red cross” or “white dash”, from the borders of its ::before and ::after, leaving out parts that draw nothing (Req. 14, 23, 26).

12.27 `simpleSpot()` EXPORTED

Returns where the Simple UI box is, to check that switching Simple UI does not move it (Rule 18).

12.28 `simpleTicked(on)` EXPORTED

Tells whether the Simple UI box shows the given state, for the eye and for screen readers.

12.29 `civLinks()` EXPORTED

Counts the civilisations in the list that are links.

12.30 `focusables()` EXPORTED

Lists every element that can be seen and reached with the keyboard.

12.31 `textOf(node)` INTERNAL

Reads an element's text as a screen reader would, leaving out parts marked aria-hidden.

12.32 `tidy(text)` EXPORTED

Turns runs of spaces into single spaces and trims the ends.

12.33 `nameFor(el)` EXPORTED

Works out an element's accessible name much as a browser does: from aria-labelledby, aria-label, its label or its text. Lets the tests check that controls are named (WCAG 4.1.2).

12.34 `squash(text)` EXPORTED

Lower-cases a text and keeps only letters and digits, for loose comparisons.

12.35 `visibleTexts(el)` EXPORTED

Lists the visible pieces of text inside an element. Used to find stray placeholders, and to compare names with what can be seen (WCAG 2.5.3).

12.36 `keyDown(target, key)` EXPORTED

Sends a key press to an element, as a visitor's keyboard would.

12.37 `tap(el)` EXPORTED

Clicks an element with mouse pointer events, as a visitor would.

12.38 `type(text)` EXPORTED

Types a text into Find, as a visitor would.

12.39 `pick(code)` EXPORTED

Chooses a language through the Options window and its menu, as a visitor would.

12.40 `reset()` EXPORTED

Returns the page to its start: Find empty, no marks, every branch closed, the menu and the Options window closed, nothing focused. Tests start from here.

12.41 `beside()` EXPORTED

Tells whether Find's message is on one line, right of the box and level with it (Req. 21).

12.42 `listed()` EXPORTED

Lists the civilisations the page shows.

12.43 `expected(set)` EXPORTED

Lists the names a set of civilisations should show, translated and sorted, to compare with listed().

12.44 `below(item, top)` EXPORTED

Tells whether an item sits anywhere under another.

12.45 `itemAbove(item)` EXPORTED

Tells whether an item has another item above it, which makes it a unit or technology. The tests work this out on their own, apart from the page's code.

12.46 `hex(code)` INTERNAL

Turns a colour written as six hex digits into the rgb() form the browser reports.

12.47 `edges()` EXPORTED

Reads colour_key.csv into the outline colour of each kind and reach of tree icon, so the tests compare the page with the file.

12.48 `shownKeys()` EXPORTED

Lists the items currently in the tree, to compare the default tree with the full one.

12.49 `uniqueKeys()` EXPORTED

Lists the items inside unique groups, which the default tree leaves out.

12.50 `regionalKeys()` EXPORTED

Lists the items aoe2techtree.net calls regional, which Include Regionals leaves out when unticked.

12.51 `outside(nodes, regional)` EXPORTED

Lists the items of a tree outside regional items and all under them: what the tree shows with Include ticked and Include Regionals unticked.

12.52 `keptPath()` EXPORTED

Finds the path of a leaf to mark while an Include box changes. Paths survive the tree being built again; item objects do not.

12.53 `byPath(path)` EXPORTED

Finds the item with a path in the tree as it is now, after it was built again.

12.54 `onTop(item)` INTERNAL

Tells whether an item is the top of its line: a unit or technology right under a building or group. The tests work this out on their own, apart from the page's code (Req. 27).

12.55 `lines()` INTERNAL

Lists the items that start a line.

12.56 `inner(item)` INTERNAL

Tells whether an item is a unit or technology below the top of its line with one item under it, where a mark passes down only at the same count (Req. 29, 30).

12.57 `fixtures()` EXPORTED

Picks sample items from the current tree: leaves some civilisations have, buildings, the top of a line, even and uneven counts, a building some do without. Found from the data, so tests survive game updates.

13 js/tests/static.js

Tests that read the files themselves, not the page.

13.1 `scripts()` EXPORTED

Finds every script by following the imports from script.js, noting which load only on demand, so the rule checks cover new scripts by themselves.

13.2 `linesOfCode(code)` INTERNAL

Counts the lines that are not blank, which is how Rules 23 and 25 count.

13.3 `regexMayStart(last, word)` INTERNAL

Tells whether a slash after a given character or word starts a regular expression rather than a division, so blanked() can skip its contents.

13.4 `closing(code, at)` INTERNAL

Finds where a string or regular expression that starts at a position ends, skipping escaped characters.

13.5 `blanked(code)` INTERNAL

Blanks out the contents of strings, template texts and regular expressions, keeping line breaks, so braces inside them cannot confuse the count of functions.

13.6 `opensBody(text, at, paren)` INTERNAL

Tells whether a brace opens a function's body: after =>, or after the parameters of a function rather than an if, for, while, switch or catch.

13.7 `functionsOf(code)` INTERNAL

Finds every function body with braces in a script, by pairing each body's opening brace with its closing one.

13.8 `longFunctions(code, lines)` INTERNAL

Lists the functions over 60 lines of code (Rule 23), with their names and lines.

13.9 `sizeProblems(page, sheet, files)` INTERNAL

Lists every file over 400 lines of code (Rule 25), function over 60 (Rule 23), and script with lines over 120 characters (Rule 24).

13.10 `badImages(page, sheet)` INTERNAL

Tells whether the sources show an image they may not: media in the page, <img> outside templates, a tab icon before the scripts, or a url() outside .pictures rules and @font-face.

13.11 `sourceTests(page, sheet, files)` EXPORTED

Checks the files against the rules: no comments, no CSS or HTML in scripts, where scripts live, their sizes, which scripts touch the page, images only for the pictures, no game data in code.

13.12 `textTests()` EXPORTED

Checks that every language has every page text and group name, and a name for every item and civilisation. Part of the functional round.

13.13 `tagTests()` EXPORTED

Checks that every language has a name and a valid language tag (WCAG 3.1.1, 3.1.2). Part of the accessibility round.

13.14 `treeTests(all)` INTERNAL

Checks the tree in data.json: each item once, no fields but item, group, kids and unique, unique only on groups, and each item's availability and icon.

13.15 `textDataTests(all)` INTERNAL

Checks the texts in data.json: plural forms, the same placeholders in every language, overrides of known names, and the source, tool texts and ignore list.

13.16 `dataTests()` EXPORTED

Checks data.json for a maintainer's mistakes, through treeTests() and textDataTests(). Part of the maintainability round.

13.17 `updaterTests()` EXPORTED

Checks the updater's writer: the files read back without loss, who has what is stored in its shortest form, and names are already tidy, so an update changes only what really changed.

14 js/tests/behaviour.js

Tests of the tree: marking, Filters, counts, opening and closing, the keyboard, and Find.

14.1 `chainOf(item)` INTERNAL

The tests' own version of a branch's chain of only children, kept apart from the page's, so a mistake there cannot hide itself.

14.2 `chipNames()` INTERNAL

Lists the names on the Filters buttons.

14.3 `fullName(item)` INTERNAL

Returns “Full” and an item's name, as Filters names a fully marked building (Req. 24).

14.4 `marking(sample)` INTERNAL

Tests marking by click, X, the menu key and Space, the civilisation list and its count, and the first Filters button (Req. 3, 4, 22).

14.5 `branches(sample)` INTERNAL

Tests buildings, groups and lines: what they mark and show, their Filters names, “Full” only with something under it (Req. 24, 25), and the green tick, red cross and white dash (Req. 14, 23, 26).

14.6 `crowding()` INTERNAL

Tests that a full tree and a long civilisation list never make the page scroll, crush a pane or overlap, in whatever window the tests run (Rule 12).

14.7 `filterFocus(sample)` INTERNAL

Tests that clearing a filter moves focus to the next one, and from the last one to the heading, with the hint shown (WCAG 2.4.3). Accessibility round.

14.8 `gestures(sample)` INTERNAL

Tests that a long press with a finger marks must lack, and that a right-click clears it (Req. 3).

14.9 `wholeLines(sample)` INTERNAL

Tests that marking the top of a line marks the whole line whatever the counts, must have or must lack, that marking it again clears it, and that a line's end marks only itself (Req. 27, 30).

14.10 `parents(sample)` INTERNAL

Tests that a mark never passes up: marking everything at a building's ends leaves the building unmarked, and a building can be must lack over must have items (Req. 30).

14.11 `lines(sample)` INTERNAL

Tests that marking a unit or technology opens it to its first fork or end, and that below the top of a line a mark passes down only while the number of civilisations stays the same (Req. 28 to 31).

14.12 `counts(sample)` INTERNAL

Tests the numbers on the items: closed and open, on lines, and with filters (Req. 3).

14.13 `tree()` INTERNAL

Tests Expand all, Collapse all and the arrows, and that opening a branch opens its only children (Req. 18, 19).

14.14 `keyboard(sample)` INTERNAL

Tests moving through the tree with the arrow keys, Home and End, and opening a line with Right (WCAG 2.1.1). Accessibility round.

14.15 `finding(sample)` INTERNAL

Tests that Find highlights its matches in yellow, opens to and counts them, and says when nothing matches (Req. 20, 21).

14.16 `findKey(sample)` INTERNAL

Tests that Enter in Find jumps to the first match (WCAG 2.1.1). Accessibility round.

14.17 `behaviourTests(sample)` EXPORTED

Runs the tree's functional tests, in order.

14.18 `keyboardTests(sample)` EXPORTED

Runs the tree's accessibility tests: focus after clearing filters, the keyboard, and Enter in Find.

15 js/tests/interface.js

Tests of the language menu, the languages and the Options window, and the runs of the accessibility audit.

15.1 `englishFirst()` INTERNAL

Tells whether a first visit starts in English, with no saved choice and a German browser (Req. 10), then puts the saved choice back.

15.2 `menuTests(sample)` EXPORTED

Tests the language menu: in Options, every language, closing on Escape, a click or leaving it, a window that never resizes, the language changed and kept, English first, marks kept (Req. 8 to 12).

15.3 `languageTests(code, sample)` EXPORTED

Tests one language: the page's text, the tree's names, no placeholders, the footer's links and Microsoft's notice, and a translated, sorted civilisation list (Req. 4, 5, 11).

15.4 `optionTests()` EXPORTED

Tests the two Include boxes: what the tree leaves out, ticking and unticking keeping marks, the choices remembered, and Options opening and closing properly (Req. 7, 13 to 16).

15.5 `menuAudit(sample)` INTERNAL

Runs the accessibility audit with the language menu open. This is the audit's first run, so its checks are listed from here.

15.6 `menuKeys()` INTERNAL

Tests the language menu's keys: Down opens it at the current language, and Down and End move through it (WCAG 2.1.1).

15.7 `languageAudits(code, sample)` INTERNAL

Runs the accessibility audit in one language: as loaded, with filters, and with everything open. The focus checks run in the first language.

15.8 `spacingTests(sample)` INTERNAL

Tests that wider text spacing, as WCAG 1.4.12 describes, makes nothing stick out sideways.

15.9 `optionKeys()` INTERNAL

Tests that Space unticks Include and Include Regionals, leaving those items out and keeping marks, and ticks them again (WCAG 2.1.1).

15.10 `optionAudits()` INTERNAL

Runs the audit with the Options window open, and with everything included and open.

15.11 `accessibilityTests(sample)` EXPORTED

Runs this file's accessibility tests in the order TESTS.md lists them, then the remaining audits.

16 js/tests/pictures.js

Tests of the pictures, run with Simple UI off; there the page need not meet WCAG (Rule 22), nor the UI particulars of REQUIREMENTS.md (Req. 0).

16.1 `sharp()` INTERNAL

Tells which pictures the screen should get: 4k when the window is bigger than 1920 by 1080 or the screen is sharp, sd otherwise.

16.2 `at(url, path)` INTERNAL

Tells whether a CSS url() points at a given picture in assets/images/.

16.3 `picture(style, path)` INTERNAL

Tells whether an element's background is a given picture.

16.4 `offParchment(style)` INTERNAL

Lists the header, panes and footer that stick out of the parchment, worked out from the five numbers in style.css and the window's size.

16.5 `strayCursors()` INTERNAL

Lists the elements without the right cursor: the pointing hand over links, the sword elsewhere, x1 or x2 by the window's size.

16.6 `pictureLooks()` INTERNAL

Tests the pictures as a whole: background, cursors, the Options button and its ring, the tab icon, the parchment and the footer.

16.7 `wrongBoxes()` INTERNAL

Lists the checkboxes without the sheet's picture for their state, empty, ticked or crossed, or with a drawn mark other than the dash, the radio buttons without theirs, and any state with no box.

16.8 `unpictured()` INTERNAL

With every item included, lists the tree rows without the right icon: its file, loaded, 44 px, between box and name, and outlined as colour_key.csv says.

16.9 `findLooks()` INTERNAL

Tests the Find box with the pictures: its frame, search icon and clearing cross, its width of up to 325 px, the message beside it, and that matches are marked (Req. 21).

16.10 `hover(el, x, y)` INTERNAL

Moves the mouse over an element, as a visitor would.

16.11 `tipLooks()` INTERNAL

Tests the tooltip: over the tree at the cursor's corner, over Include with its description, which then shows only there, and none when Tooltips is unticked.

16.12 `simpleSwitch()` INTERNAL

Ticks and unticks Simple UI, and lists what went wrong: a picture, tab icon or link left, the box moved, the choice not remembered, or something not back.

16.13 `pictureTests()` EXPORTED

Runs the tests of the pictures with Simple UI off, then turns it on again (Rules 9, 18, 22; Req. 0, 32, 33).

17 js/tests/audit.js

The accessibility audit: checks the page as it stands against WCAG 2.2 AAA, wherever a test calls it.

17.1 `rgba(value)` INTERNAL

Reads a CSS colour into its red, green, blue and alpha numbers.

17.2 `blend(top, bottom)` INTERNAL

Blends a see-through colour over the colour below it, to get the colour actually seen.

17.3 `backgroundOf(el)` INTERNAL

Works out the colour behind an element by blending its own and its ancestors' backgrounds. Contrast is measured against this.

17.4 `luminance(rgb)` INTERNAL

Computes a colour's relative luminance, as WCAG defines it.

17.5 `contrast(a, b)` INTERNAL

Computes the contrast ratio of two colours, as WCAG defines it, from 1:1 to 21:1.

17.6 `contrastOn(colour, behind)` INTERNAL

Computes the contrast of a colour, which may be see-through, over the colour behind it.

17.7 `visibleCentre(el)` INTERNAL

Finds the middle of the visible part of an element's first line, clipped by every scrolling area around it, to see what is drawn there.

17.8 `focusChecks(list)` INTERNAL

Focuses every control in turn, and checks that it shows a thick outline with 3:1 contrast and that nothing covers it (WCAG 2.4.7, 2.4.12, 2.4.13).

17.9 `structureChecks(note, list)` INTERNAL

Checks the page's title and languages, its landmarks and headings, and that ids are unique and ARIA references resolve (WCAG 1.3.1, 2.4.2, 3.1.1, 3.1.2, 4.1.2).

17.10 `controlChecks(list)` INTERNAL

Checks that boxes, branches and the language menu say their state, and that every control has a name containing its visible text (WCAG 2.5.3, 4.1.2).

17.11 `colourChecks(all, list)` INTERNAL

Checks the contrast of all visible text, 7:1, or 4.5:1 when large, and of control borders, 3:1 (WCAG 1.4.6, 1.4.11).

17.12 `layoutChecks(all, note, list)` INTERNAL

Checks 44 by 44 targets, no page scrolling, nothing sticking out sideways, and unjustified text with 1.5 line spacing (WCAG 2.5.5, 1.4.10, 1.4.8; Rule 12).

17.13 `mediaChecks(all, list)` INTERNAL

Checks that the page shows no media, tab icon or CSS picture, and runs no animations or transitions (Rules 9, 10, 11, 22).

17.14 `audit(label, options)` EXPORTED

Checks the page as it stands against WCAG 2.2 AAA through the checks above, and the focus checks if asked. Called in many states and languages.

18 data.json

Everything the page knows, in English: the data source and Microsoft's notice, the languages, the civilisations, where the icons are and the tab icon, the tree of the left pane, which civilisations have each item and each item's icon, items left out on purpose, corrections to names, the texts of the updater and the tests, and the page's text. The updater rewrites it; the tree, the icons, the ignore list and the overrides are edited by hand.

```
{"source": {…}, "languages": {…}, "civs": […], "icons": {…}, "tree": […],
 "availability": {…}, "pictures": {…}, "ignore": […], "overrides": {…},
 "tools": {…}, "text": {"ui": {…}, "groups": {…}, "civs": {…}, "items": {…}}}
```

19 locales/de.json (example)

One file per other language; this one is German. It holds the language's corrections to names (overrides) and its text: the page's texts, group names, and civilisation and item names. The updater fills in the names; the page's texts are translated by hand. Only the language shown is downloaded.

```
{"overrides": {"Composite Bowman": "Kompositbogenschütze", …},
 "text": {"ui": {"language": "Sprache", "clear": "Zurücksetzen", …},
 "groups": {…}, "civs": {…}, "items": {…}}}
```